BioEval: An LLM-Driven Framework for Evaluating Dataset Transparency, Reproducibility, and Information-Theoretic Rigor in Computational Simulation Papers

Joel Dietz (ORCID 0009-0007-4948-9192)

June 6, 2026

Abstract

Computational and agent-based simulation papers increasingly drive claims about biological coordination — ant and termite stigmergy, honeybee colony dynamics, firefly synchronization, and ant-colony optimization — yet their reproducibility is rarely assessed systematically. Two failures recur: datasets and code are declared without resolvable, verifiable provenance, and systems whose entire subject is information flow are described mechanistically without ever quantifying that information. BioEval is an LLM-driven evaluation framework that scores a paper on a versioned seven-dimension rubric. Six dimensions measure conformance to good data and reproducibility practice — data disclosure, dataset resolvability, code availability and versioning, code-to-data traceability, simulation derivation clarity, and reproducibility-package quality. A seventh, orthogonal dimension scores Information-Theoretic Rigor: whether the paper formalizes and quantifies the information content of the system it models (Shannon entropy, mutual and transfer entropy, channel capacity, communication bit rate). Scores are grounded in verified external evidence — resolved accessions and DOIs, live repository facts — rather than the paper's own claims. We apply BioEval (rubric v0.9.0) to a corpus of 14 insect-swarm and agent-based simulation projects. Transparency is uneven and reproducibility packaging is consistently weak; most strikingly, information-theoretic rigor is systematically low even though communication and coordination are the modeled phenomena. This deposition bundles the report together with the complete, MIT-licensed source code of the evaluation system.

Keywords: reproducibility · dataset transparency · information theory · agent-based simulation · stigmergy · swarm intelligence · code-to-data traceability · large language models · FAIR data

1 Introduction

Agent-based and computational simulations are now a primary vehicle for claims about biological coordination: how ant and termite colonies self-organize through stigmergy, how honeybee colonies allocate labor, how populations of fireflies synchronize, and how ant-colony optimization transfers these mechanisms to engineering. The credibility of such claims rests on two things that are easy to assert and hard to verify. First, the data and code behind a simulation must be disclosed in a way that a reader can actually resolve and re-run — declaring that a dataset 'is available' is not the same as providing a resolvable accession, a versioned repository, and a runnable package. Second, when the modeled system is itself a communication system, scientific rigor demands that the information it carries be formalized and quantified, not merely narrated.

BioEval addresses both gaps in a single instrument. It is an evaluation framework that submits a paper (by URL or PDF), extracts its real text, gathers external evidence, and uses a large language model to score the paper against a fixed, versioned rubric. The framework deliberately separates two orthogonal questions: does the work conform to good data and reproducibility practice, and — independently — does it bring information-theoretic rigor to a system whose very subject is information flow? This report explains how the system works and presents its evaluations of a corpus of insect-swarm and agent-based simulation projects.

2 The BioEval Framework

2.1 Pipeline

A submission is fetched through an SSRF-guarded path and its text is extracted from the real PDF (via a pure-JavaScript pdf.js build); if too little text can be recovered — for example a scanned, image-only document —

the evaluation errors out with a stated reason rather than scoring garbage. The extracted text is then analyzed by Anthropic's Claude in an evidence-gathering pass that resolves declared identifiers against Crossref and DataCite and inspects any linked code repository for live facts (license presence, releases, structure). Only after evidence is collected does the model assign the seven dimension scores together with a structured set of findings, gaps, and recommendations, all persisted and aggregated in a dashboard.

2.2 The seven dimensions

The rubric is versioned (currently v0.9.0, intentionally pre-1.0 while it is validated by reviewers) and stamped onto every evaluation so scores remain interpretable as the rubric evolves. Each dimension is scored 0–100 against fixed guideposts; the overall score is the weighted mean of all seven, using the weights below.

Dimension	Weight	What it tests
Data Disclosure	18%	Whether the paper names the data it uses and declares its provenance, access conditions, and licensing — not merely asserting that data exists.
Dataset Resolvability	14%	Whether declared datasets carry resolvable identifiers (accessions, DOIs, repository URLs) that actually resolve to the stated record.
Code Availability & Versioning	14%	Whether the simulation code is publicly archived, versioned, and reachable at a stable location, with an explicit license.
Code-to-Data Traceability	18%	Whether each computational step can be traced back to the specific data source and citation it depends on.
Simulation Derivation Clarity	18%	Whether model parameters, assumptions, and update rules are derived transparently rather than asserted, so the simulation can be re-derived.
Reproducibility Package Quality	8%	Whether a runnable package (environment spec, seeds, instructions) lets an independent reader reproduce the reported results.
Information-Theoretic Rigor	10%	Whether the paper formalizes and quantifies the information content of the system it models. Orthogonal to transparency; non-applicable topics are scored at a neutral baseline (~50).

2.3 Conformance to good data and reproducibility practice

Six of the seven dimensions measure whether a simulation accords with established good practice for data and software. Data Disclosure and Dataset Resolvability test the FAIR ideal that data be findable and accessible through identifiers that genuinely resolve — a declared dataset with no resolvable accession scores poorly even if the prose claims openness. Code Availability & Versioning and Reproducibility Package Quality test whether the software is archived, licensed, versioned, and packaged so an independent reader can run it. Code-to-Data Traceability and Simulation Derivation Clarity test the chain of custody from result back to source: can each computational step and model parameter be traced to the specific data and citation it depends on, rather than asserted? Crucially, these scores are anchored to verified external signals — a resolved DOI, a real license file detected in the repository — and not to the paper's self-description; the framework does not credit unverifiable claims, and does not invent gaps it cannot substantiate.

2.4 Code-to-data traceability

Beyond scoring, BioEval can accept the simulation's source code directly: the model segments the code and maps each segment to the data source and citation it relies on, with a high/medium/low confidence rating. This makes the dependency between a result and its inputs explicit and auditable, which is the operational meaning of reproducibility for a simulation.

3 Information-Theoretic Rigor

The seventh dimension is orthogonal to the other six: it measures scientific-content rigor rather than transparency. Many papers in this domain model communication and coordination systems — pheromone stigmergy in ants and termites, waggle-dance signaling in bees, phase coupling in firefly and Kuramoto

synchronization — where information flow is not a side effect but the phenomenon itself. For such systems, rigor means formalizing that information: the Shannon entropy of agent state or signal distributions, the mutual or transfer entropy that captures directed information flow between agents, and the channel capacity or communication bit rate of the signaling mechanism (bits per pheromone deposit, bits per dance, bits per second), including how these scale with colony size and signal-to-noise.

The guideposts reward papers that actually do this mathematics and penalize papers where communication is central yet treated purely mechanistically. To avoid punishing work for which information theory is simply not applicable — a pure phylogenetics or sequence-alignment pipeline, say — such topics are scored at a neutral midpoint (~50) rather than zero, so non-applicability neither rewards nor punishes. Because the dimension is orthogonal to transparency, a paper can be exemplary in data practice yet score low here, and that contrast is itself the finding this report foregrounds.

4 Evaluation Corpus and Method

We evaluated 14 computational-simulation projects spanning ant, bee, termite, and firefly systems and ant-colony optimization, drawn from the insect-swarm simulation series. Every project was scored under a single rubric version (v0.9.0) so the results are mutually comparable. Scores are assigned by the language model against the fixed guideposts of Section 2, grounded in the external evidence gathered during analysis. In the table below the overall column is recomputed as the exact weighted mean of the seven dimensions, so each row is reproducible from the published weights.

5 Results

5.1 Score summary

Simulation project	Ovr	DD	DR	CA	TR	SC	RP	IT
BeeStack: An Evidence-Typed Scaffold for Whole-Colo…	**55.6**	72	68	63	70	58	42	30
Bee Swarm Simulation	**53.0**	72	68	58	62	42	55	32
The Ant Stack: Reproducible scientific workspace fo…	**44.7**	52	48	62	55	42	58	22
Ant Simulation — Pygame Stigmergy	**40.4**	62	58	63	45	28	42	12
Ant Colony Optimization — React/Canvas	**39.2**	52	55	72	38	22	62	12
Rust Ant Colony Simulation	**39.0**	62	70	52	35	28	48	10
Locas Ants — Lua/Love2D Ant Colony	**37.1**	42	48	72	30	22	55	20
ACO Robot Path Planning (Python)	**28.6**	22	38	48	35	28	12	20
Firefly Synchronization Simulation (JS)	**28.0**	28	35	48	30	22	25	18
Hardware-Accelerated Ant Colony Swarm (C++)	**27.9**	28	35	52	22	18	35	20
Termite Colony — Unity 3D Multiagent	**27.5**	42	38	38	22	30	22	12
Ants Sandbox (TypeScript/web)	**23.2**	22	28	62	15	12	38	10
Symbiants — Ant Colony Sim (Rust)	**22.1**	18	30	58	12	15	35	10
swarm-abc — Artificial Bee Colony Simulation	**18.6**	18	22	32	12	18	8	18

DD Data Disclosure · DR Dataset Resolvability · CA Code Availability · TR Code-to-Data Traceability · SC Simulation Clarity · RP Reproducibility Package · IT Information-Theoretic Rigor. Overall is the weighted mean of all seven dimensions. Scores are rubric v0.9.0 (0–100).

5.2 Observations

Three patterns recur across the corpus. Transparency is uneven: code availability tends to be the strongest dimension — most projects publish a repository — while reproducibility-package quality and code-to-data traceability are consistently the weakest, meaning the code exists but cannot easily be re-run or traced to its inputs. The headline finding is the systematically low Information-Theoretic Rigor across projects whose entire subject is communication and coordination: even strong, transparent simulations rarely quantify the

information their agents exchange. The orthogonality of the rubric makes this visible — high transparency and low information-theoretic rigor coexist in the same papers, marking a concrete and addressable opportunity for the field.

5.3 Per-project synopsis

BeeStack: An Evidence-Typed Scaffold for Whole-Colony Honeybee Simulation — overall 55.6 (IT 30)

BeeStack demonstrates commendable transparency infrastructure: the software is deposited on Zenodo with a DOI and versioned GitHub release, and the repository source files contain numerous resolved DOIs linking to empirical datasets (e. g.

Principal gap: GitHub API detects NO license file in the repository itself, contradicting the MIT declaration on Zenodo; this creates a legal ambiguity for reuse directly from GitHub

Bee Swarm Simulation — overall 53.0 (IT 32)

This is a browser-based agent simulation with all source files publicly available on GitHub and no external build dependencies, making it straightforwardly runnable for anyone with a browser. The citation-backed mode documents its biological parameter sources against a credible set of published references (von Frisch 1967, Seeley 1995, Couvillon 2019, Schurch 2021, Menzel 2023), but the heuristic mode's hand-tuned parameter values are not documented, and neither mode provides pinned numerical parameter tables traceable to specific passages.

Principal gap: No software license is present in the repository, creating legal ambiguity around reuse and reproduction

The Ant Stack: Reproducible scientific workspace for ant-inspired simulation and complexity energetics — overall 44.7 (IT 22)

The Ant Stack presents a modular simulation framework with a reasonably structured GitHub repository (antstack_core), a passing test suite (676 tests), documented CLI entry points, manifest-driven artifact generation with SHA-256 checksums, and YAML-based configuration files — all positive transparency signals. However, the paper is primarily a design specification and roadmap rather than a completed computational study: key contributions (species presets, evaluation benchmarks, baseline seeds) are described as future work, no public repository URL is provided, no archived DOI for the code exists, and foundational datasets (FORMINDEX, MetaInformAnt, Blue Brain, Eyewire) are cited without accession numbers or data availability statements.

Principal gap: No public URL for the GitHub repository is provided in the paper or README, making direct discovery by readers non-trivial

Ant Simulation — Pygame Stigmergy — overall 40.4 (IT 12)

This is a publicly accessible GitHub simulation repository with an MIT license, a minimal README, and a requirements. txt dependency manifest, making it partially reproducible.

Principal gap: Dependency versions are not pinned in requirements.txt, making exact environment reconstruction uncertain across future Python/library versions

Ant Colony Optimization — React/Canvas — overall 39.2 (IT 12)

This project is a self-contained React/Canvas ant simulation with no external datasets, and its generative inputs (JSON map, sprite sheets, tileset) are all bundled within the public GitHub repository under an MIT license with installation instructions provided. However, reproducibility is weakened by a placeholder clone URL, absence of a pinned commit or versioned release, no surfaced dependency versions in the README, and hard-coded simulation parameters (ant states, movement rules) with no cited sources or documented rationale.

Principal gap: No pinned commit hash or tagged release is cited, so the exact code state cannot be unambiguously referenced

Rust Ant Colony Simulation — overall 39.0 (IT 10)

This is a public GitHub repository for an ant colony simulation written in Rust using the Bevy game engine, with no external datasets (appropriately so for a generative simulation). The code is publicly accessible with a Cargo.

Principal gap: No software license is present, creating legal ambiguity for reuse or reproduction

Locas Ants — Lua/Love2D Ant Colony — overall 37.1 (IT 20)

This is a GitHub-hosted Lua/Löve2D ant colony simulation with 161 commits, 6 versioned releases, an MIT license, and a basic README providing installation instructions — a reasonable open-source release but far below research-grade reproducibility standards. Because the project uses no external biological or experimental datasets, evaluation pivots to disclosure of generative inputs (rules, parameters, initial conditions), which are entirely embedded in source code with no documentation, configuration files, or parameter tables provided in the README or supplementary materials.

Principal gap: Simulation parameters (pheromone evaporation rate, deposit amounts, ant sensing radius, colony size, movement rules) are embedded in source code with no externalized configuration file, parameter table, or documentation

ACO Robot Path Planning (Python) — overall 28.6 (IT 20)
This repository provides a publicly accessible Python implementation (PathPlanning. py) of an ACO-based robot path planning algorithm, licensed under GPL-3.

Principal gap: No dependency manifest (requirements.txt, setup.py, or equivalent) is present, making environment reconstruction manual and error-prone

Firefly Synchronization Simulation (JS) — overall 28.0 (IT 18)
This is a GitHub-hosted firefly synchronization simulation with publicly accessible code but extremely limited reproducibility infrastructure: no license, no versioned release or pinned commit, a minimal README (~42 bytes), no CI/tests, and no archived snapshot (e. g.

Principal gap: No software license detected, making legal reuse and redistribution ambiguous

Hardware-Accelerated Ant Colony Swarm (C++) — overall 27.9 (IT 20)
This repository presents a hardware-accelerated ant colony swarm simulation using CUDA and OpenGL, with source code publicly available on GitHub (12 stars, 38 commits, last pushed May 2026). However, the repository lacks a license, version tags, releases, dependency manifests with pinned versions, CI/CD, and tests, and the README is minimal (~1264 bytes) with no documentation of swarm parameters, evolutionary algorithm configuration, grid size, agent counts tested, or initial conditions.

Principal gap: No software license is present in the repository, preventing legal reuse or redistribution

Termite Colony — Unity 3D Multiagent — overall 27.5 (IT 12)
This is a GitHub README-only submission for a Unity/C# termite colony simulation with no associated research paper, journal article, or formal methods section. The repository is public under MIT license with 38 commits but has no versioned releases, no dependency manifest, no CI, no tests, and a README that was flagged as missing/empty by the live GitHub API — contrasting with the scraped content provided.

Principal gap: The clone URL in the README (https://github.com/Dougarasu/TrabalhoIA.git) does not match the actual repository URL, breaking reproducibility for anyone following instructions literally

Ants Sandbox (TypeScript/web) — overall 23.2 (IT 10)
This repository is a browser-based ant colony simulation written in TypeScript/Vue, publicly available on GitHub under MIT license with basic build instructions, but it falls short on nearly every scientific reproducibility dimension. There are no external datasets (appropriate for a simulation), but the generative parameters, agent rules, pheromone decay constants, and initial conditions are entirely undocumented — the README is a minimal ~490-byte stub with no parameter descriptions or citations.

Principal gap: Simulation parameters (pheromone decay rates, evaporation constants, ant sensing radii, colony size, food quantities, etc.) are not documented anywhere in the README or linked configuration files

Symbiants — Ant Colony Sim (Rust) — overall 22.1 (IT 10)
This submission is a GitHub repository for a Rust/Bevy ant colony simulation game, not a scientific paper in the conventional sense — it lacks any scientific manuscript, methodology section, or quantitative analysis. The repository is publicly accessible under dual Apache-2.

Principal gap: No published versioned releases (0 releases confirmed via GitHub API) and no git tags, making it impossible to cite a specific reproducible snapshot

swarm-abc — Artificial Bee Colony Simulation — overall 18.6 (IT 18)
This repository contains a bare-minimum public GitHub repository for an Artificial Bee Colony simulation implemented in HTML/JavaScript/CSS, but it lacks virtually all elements required for scientific reproducibility. There is no README, no license, no dependency manifest, no versioned release or tagged commit, no documented parameters, no CI, and no archived snapshot (e.

Principal gap: No README file is present, providing zero guidance on how to run, configure, or interpret the simulation

6 Discussion

BioEval treats reproducibility as a property that must be demonstrated through resolvable evidence rather than claimed in prose, and it adds a second axis — information-theoretic rigor — that is specific to the communication systems this literature studies. The two axes are complementary: good data practice makes a result checkable, while information-theoretic formalization makes it meaningful for systems whose subject is information. Using a language model as the evaluator is what makes assessment at this breadth tractable; fixed guideposts, evidence grounding, and a versioned rubric are what keep it disciplined.

7 Limitations

Scores are produced by a language model and tend to snap to the discrete tiers defined by the guideposts; feeding real, per-item external signals (resolved accessions, live repository facts) is what differentiates otherwise-similar papers. The rubric is pre-1.0 and still under reviewer validation, so weights and guideposts may change between versions — hence the stamped rubric version on every evaluation. The corpus is domain-specific (insect-swarm and agent-based simulation), and the information-theoretic dimension is calibrated for systems where information flow is central; its neutral-baseline treatment of non-applicable topics is a deliberate, conservative choice rather than a measurement.

8 Availability and Reproducibility

BioEval is released under the MIT license. The source repository is hosted at github.com/fractastical/bioinformatics-eval, and this Zenodo deposition bundles the present report together with a complete archive of the source code at the corresponding revision. The system is built as a pnpm monorepo (React + Vite frontend, Express 5 API, PostgreSQL with Drizzle ORM, contract-first OpenAPI/Orval codegen) and uses Anthropic Claude through Replit's AI integrations. The evaluation rubric and the software are co-versioned at v0.9.0.

References

[1] Shannon, C. E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal, 27, 379–423, 623–656.

[2] Grassé, P.-P. (1959). La reconstruction du nid et les coordinations interindividuelles: la théorie de la stigmergie. Insectes Sociaux, 6, 41–80.

[3] Dorigo, M., & Stützle, T. (2004). Ant Colony Optimization. MIT Press.

[4] Kuramoto, Y. (1984). Chemical Oscillations, Waves, and Turbulence. Springer.

[5] Schreiber, T. (2000). Measuring Information Transfer. Physical Review Letters, 85(2), 461–464.

[6] Wilkinson, M. D., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. Scientific Data, 3, 160018.

[7] Sandve, G. K., et al. (2013). Ten Simple Rules for Reproducible Computational Research. PLoS Computational Biology, 9(10), e1003285.